

Pipeline Guide

دليل خط الأنابيب الكامل

From Raw Monthly Contracts to Hybrid Trading Model
من ملفات العقود الشهرية الخام إلى النموذج الهجين

6B (GBP/USD Futures) — End-to-End Operational Guide

عقود الجنيه الإسترليني — دليل تشغيل من البداية للنهاية

Table of Contents

فهرس المحتويات

1. Overview & Prerequisites

نظرة عامة + المتطلبات

2. Step 1 — Smart Contract Extraction (Rollover)

الخطوة 1 — استخراج العقد الذكي (Rollover)

3. Step 2 — Per-Month prepare_day_trading

الخطوة 2 — تشغيل prepare_day_trading لكل شهر

4. Step 3 — Build Order Batches

الخطوة 3 — بناء Order Batches

5. Step 4 — Combine Quarterly / Yearly Artifacts

الخطوة 4 — دمج الأرباع / السنوات

6. Step 5 — Quality Checks after Merge (CRITICAL)

الخطوة 5 — فحوصات الجودة بعد الدمج (حرجة)

7. Step 6 — SSL Pretraining

الخطوة 6 — التدريب السببي SSL

8. Step 7 — Train Hybrid Model

الخطوة 7 — تدريب النموذج الهجين

9. Step 8 — Strict Backtest & Decision Policy

الخطوة 8 — Backtest صارم + سياسة القرار

10. Appendix — Troubleshooting & Common Issues

ملحق — استكشاف الأخطاء

1. Overview & Prerequisites

١. نظرة عامة + المتطلبات

What this pipeline does

ما يفعله هذا ال pipeline

Takes 6 years of raw monthly Databento files (each containing both near-month and far-month contracts) and produces: (a) a continuous back-adjusted dataset, (b) 161-dim SSL embeddings per bar, and (c) a trained HybridModel that fuses rule labels + SSL into per-bar trading decisions with adaptive TP and regime-aware sizing.

مدرب يدمج labels القواعد + SSL إلى قرارات تداول لكل bar مع TP تكييفي وتحديد حجم مبني على ال regime. (أ) داتا continuous معدلة ب Panama، (ب) embeddings 161-dim لكل bar من SSL، (ج) نموذج هجين يأخذ بيانات ٦ سنوات من ملفات Databento الشهرية (كل ملف فيه العقد القريب + العقد البعيد) وينتج:

Prerequisites

المتطلبات قبل البدء

(1) Repository on branch claudetask-d-RcDhu, latest commit. (2) Python 3.11 with the requirements installed. (3) Disk space: ~200GB for raw 6-year data + ~50GB for derived artifacts. (4) GPU recommended for SSL pretraining (CPU works but 5-10x slower).

ل ٦ سنوات + ٥٠٠ جيجا للمخرجات. (٤) GPU مفضل ل CPU SSL pretraining شغال لكن أبطأ ١٠-٥ مرات). آخر commit. (٢) Python 3.11 مع المتطلبات مثبتة. (٣) مساحة قرص: ٢٠٠ جيجا للداتا الخام (١) المستودع على فرع claudetask-d-RcDhu.

Sanity check before starting:

فحص أولي قبل البدء:

```
# 1. Verify branch + tag
git status          # → claudetask-d-RcDhu, clean
git tag -l "v*"     # → v1-baseline-pre-cleanup must exist

# 2. Verify subsystem boundaries
python tools/check_subsystem_boundaries.py
# Expected: [ ] Subsystem boundaries clean

# 3. Verify test suite passes
python -m pytest tests/ -q
# Expected: 337 passed, 2 skipped, 0 failed
```

2. Step 1 — Smart Contract Extraction (Rollover)

٢. الخطوة ١ — استخراج العقد الذكي

Goal

الهدف

Each monthly Databento file contains BOTH the near-month and the far-month contract. The extractor produces ONE continuous file per month: it uses the near contract while it's the most liquid, then switches to the far contract at the volume-crossover day, and back-adjusts the older contract's prices so there's no jump.

الأنشط، ثم يحول للعقد البعيد عند يوم تقاطع الفوليوم، ويعدل أسعار العقد القديم عشان مفيش jump في السعر. القريب + العقد البعيد. ال extractor ينتج ملف continuous واحد للشهر: يستخدم العقد القريب طول ما هو كل ملف شهري من Databento فيه العقد

Command:

الأمر:

```
# Run per monthly file (loop for 6 years × 12 months = 72 files per data type)
for year in 2019 2020 2021 2022 2023 2024; do
  for month in 01 02 03 04 05 06 07 08 09 10 11 12; do
    # MBO file (order-level)
    python tools/extract_continuous_contract.py \
      raw_data/${year}-${month}.mbo.parquet \
      --root 6B \
      --output continuous/${year}-${month}.mbo.parquet

    # MBP-10 file (depth)
    python tools/extract_continuous_contract.py \
      raw_data/${year}-${month}.mbp10.parquet \
      --root 6B \
      --output continuous/${year}-${month}.mbp10.parquet
  done
done
```

Expected output per file:

المخرج المتوقع لكل ملف:

```
Reading: raw_data/2025-03.mbo.parquet
rows: 2,001 cols: 4
```

```
Result:
mode:          continuous_with_rollover
near:          6BH5
far:           6BM5
rollover_day:  2025-03-11 00:00:00+00:00
raw_offset:    0.020005 ← carry detected automatically
applied_offset: 0.020005
output_rows:   1,500
```

```
□ continuous/2025-03.mbo.parquet (...)
□ continuous/2025-03.mbo.parquet.log.json
```

Quality checks after Step 1:

فحوصات الجودة بعد الخطوة ١:

- Every monthly file produced a `\*.log.json` with mode != 'no_rollover_picked_dominant' (rare in normal data)
- Output row count > 70% of input row count (otherwise the rollover detection is too aggressive)
- applied_offset is finite and within plausible range (for 6B: ± 0.005 typical)
- No file failed with an error — check the loop exit code

• كل ملف شهري أنتج log.json بقيمة mode مختلفة عن no_rollover_picked_dominant

• عدد rows الخارج < ٧٠٪ من المدخل (لو أقل، ال rollover صارم زيادة)

• applied_offset 6 ج قيمته منطقية (± 0.005)

• مفيش ملف فشل — تأكد من exit code لل loop

3. Step 2 — Per-Month prepare_day_trading

٣. الخطوة ٢ — prepare_day_trading لكل شهر

Goal

الهدف

Convert continuous MBO + MBP-10 monthly files into 15-min bars + 135 hand-crafted features + rule labels (event_flag, event_direction, signal_quality, path_outcome) + 9-channel LOB tensor.

تحويل ملفات MBO + MBP-10 الشهرية إلى 15 bars دقيقة + ١٣٥ feature يدوي + labels قواعد (event_flag, event_direction, signal_quality, path_outcome) + LOB tensor 9 قنوات.

```
# Run per monthly continuous file
for year in 2019 2020 2021 2022 2023 2024; do
  for month in 01 02 03 04 05 06 07 08 09 10 11 12; do
    out_dir="features/${year}-${month}"
    mkdir -p "$out_dir"

    python prepare_day_trading.py \
      --mbo continuous/${year}-${month}.mbo.parquet \
      --mbp continuous/${year}-${month}.mbp10.parquet \
      --output "$out_dir" \
      --freq 15min \
      --session_profile daytrade_default \
      2>&1 | tee "$out_dir/prepare.log"
  done
done
```

Expected outputs in each features/YYYY-MM/ dir:

المخرجات المتوقعة في كل مجلد:

```
features/2019-01/
├─ day_trading_features.parquet      # ~2000 bars × 135 cols + labels
├─ lob_tensors.npy                  # (N_bars, T=50, P=20, C=9)
├─ lob_tensor_timestamps.npy        # (N_bars,) datetime64
├─ day_trading_manifest.json        # metadata
└─ artifact_manifest.json
```

Quality checks after Step 2:

فحوصات الجودة بعد الخطوة ٢:

- Every month produced day_trading_features.parquet with N rows $\approx$ 2000-2200 (15-min bars per month)
- lob_tensors.npy first dim equals features parquet row count (alignment must be exact)
- Event rate (event_flag.sum() / len(df)) is 3-8% — too low → labeler broken, too high → false events
- regime_label distribution: trending + ranging dominate (~95%), volatile rare
- atr_14 has no NaN after warmup (first 14 bars may be NaN — acceptable)
- كل شهر أنتج day_trading_features.parquet عدد صفوف $\approx$ ٢٠٠٠-٢٢٠٠ (شموع ١٥ دقيقة شهريا)
- lob_tensors.npy أول بعد = عدد صفوف parquet (المحاذاة يجب أن تكون دقيقة)

- نسبة الحدث (event_flag) ٣-٨٪ — الأقل = labeler = مكسور، الأعلى = events = زائفة
- توزيع regime_label: trending + ranging هما الأغلب (~٩٥٪)، volatile نادر
- (مقبول — NaN أول ١٤ شمعة ممكن) warmup بعد NaN خالي من atr_14

4. Step 3 — Build Order Batches

٤. الخطوة ٣ — بناء Order Batches

Goal

الهدف

Build per-bar order tensors from raw MBO ticks: (N_bars, T=50, N_orders=200, F=7). These feed the SSL transformer alongside the LOB tensor.

ال MBO الخام: (N_bars, T=50, N_orders=200, F=7). هتدخل على SSL transformer مع LOB tensor. بناء order tensors لكل bar من ticks

```
for year in 2019 2020 2021 2022 2023 2024; do
  for month in 01 02 03 04 05 06 07 08 09 10 11 12; do
    out_dir="features/${year}-${month}"

    python self_supervised/build_order_batches.py \
      --mbo continuous/${year}-${month}.mbo.parquet \
      --features "$out_dir/day_trading_features.parquet" \
      --output "$out_dir" \
      --freq 15min \
      --lookback-bars 50 \
      --n-orders 200 \
      --tick-size 0.0001
  done
done
```

Quality checks:

فحوصات الجودة:

- order_features.npy shape: (N_bars, 50, 200, 7) — first dim matches features parquet
- order_masks.npy shape: (N_bars, 50, 200) — bool
- Sparsity (order_masks.mean()) should be 80-95% — too low means too many empty bars
- order_batches_meta.json present and parseable
 - order_features.npy شكله: (N_bars, 50, 200, 7) — عدد = bars أول بعد
 - order_masks.npy شكله: (N_bars, 50, 200) — bool
 - الكثافة (mask.mean()) يجب أن تكون ٨٠-٩٥٪ — الأقل = bars فاضية كتيرة
 - order_batches_meta.json موجود وقابل للقراءة

5. Step 4 — Combine Quarterly / Yearly Artifacts

٥. الخطوة ٤ — دمج الأرباع / السنوات

Goal

الهدف

Stitch all monthly features + LOB tensors + order tensors into ONE combined dataset (parquet + npy). Handles contract rollovers via the overlap-mode=auto detector. Marks inter-month boundaries with is_session_break=1 so the SSL filter respects them.

عبر overlap-mode=auto. يعلم الحدود بين الشهور بـ is_session_break=1. يحترمها. SSL filter. يتعامل مع rollovers العقد. dataset واحد (parquet + npy). LOB tensors + order tensors + تجميع كل ال features الشهرية

```
# Combine all 72 monthly outputs into one 6-year dataset
python self_supervised/combine_quarter_artifacts.py \
  --quarter-dirs features/2019-01 features/2019-02 features/2019-03 \
    features/2019-04 features/2019-05 features/2019-06 \
    features/2019-07 features/2019-08 features/2019-09 \
    features/2019-10 features/2019-11 features/2019-12 \
    features/2020-01 ... features/2024-12 \
  --output-dir combined_6y \
  --overlap-mode auto \
  --carry-pips-per-quarter 40
```

Expected outputs:

المخرجات المتوقعة:

combined_6y/	
└─ day_trading_features.parquet	# ~145,000 bars × 137 cols
└─ lob_tensors.npy	# (145000, 50, 20, 9) ≈ 5 GB
└─ order_features.npy	# (145000, 50, 200, 7) ≈ 40 GB
└─ order_masks.npy	# (145000, 50, 200) ≈ 1.4 GB
└─ combine_meta.json	# rollover diagnostics

6. Step 5 — Quality Checks after Merge (CRITICAL)

٦. الخطوة ٥ — فحوصات الجودة بعد الدمج (حرجة)

Why this step exists

ليه الخطوة دي مهمة

Skipping these checks is the single biggest cause of model failures. Run ALL of them after every `combine_quarter_artifacts` call. Failing any one means a data bug — fix it before SSL pretraining.

كل `combine_quarter_artifacts`. أي فحص يفشل = bug في الداتا، صلحه قبل ما تشغل SSL pretraining. تجاهل الفحوصات دي هو السبب الأول لفشل النموذج. شغل كلها بعد

(A) Alignment check — bars vs npy first-dim

(أ) فحص المحاذاة — bars مع أول بعد npy

```
python -c "  
import pandas as pd, numpy as np  
df = pd.read_parquet('combined_6y/day_trading_features.parquet')  
lob = np.load('combined_6y/lob_tensors.npy')  
of = np.load('combined_6y/order_features.npy')  
om = np.load('combined_6y/order_masks.npy')  
n = len(df)  
assert lob.shape[0] == n, f'lob first dim {lob.shape[0]} != bars {n}'  
assert of.shape[0] == n, f'of first dim {of.shape[0]} != bars {n}'  
assert om.shape[0] == n, f'om first dim {om.shape[0]} != bars {n}'  
print(f'Alignment OK ({n:}, bars on all artifacts)')  
"
```

(B) Chronological order — ts must be monotonic

(ب) ترتيب زمني — ts يجب أن يكون متصاعد

```
python -c "  
import pandas as pd  
df = pd.read_parquet('combined_6y/day_trading_features.parquet')  
ts = pd.to_datetime(df['ts_event'])  
gaps = ts.diff().dropna()  
print(f'monotonic: {(gaps >= pd.Timedelta(0)).all()}')  
print(f'min gap: {gaps.min()}')  
print(f'max gap: {gaps.max()} (weekend/roll gaps OK)')  
print(f'median gap: {gaps.median()} (should be 15min)')  
"
```

(C) Session-break markers at boundaries

(ج) علامات session_break عند الحدود

```
python -c "  
import pandas as pd  
df = pd.read_parquet('combined_6y/day_trading_features.parquet')  
n_breaks = int(df['is_session_break'].sum())
```

```

n_rolls = int(df['is_roll'].sum())
print(f'is_session_break: {n_breaks} (weekends + rolls; expect ~250-400 for 6yr)')
print(f'is_roll: {n_rolls} (contract rolls; expect ~24-30 for 6yr)')
# Show a few break ts to verify they look like Sunday opens
print('First 5 breaks:')
print(df.loc[df['is_session_break']==1, 'ts_event'].head().tolist())

```

(D) Price continuity at rollover boundaries

(د) استمرارية السعر عند حدود ال rollover

```

python -c "
import pandas as pd, numpy as np
df = pd.read_parquet('combined_6y/day_trading_features.parquet').reset_index(drop=True)
rolls = df.index[df['is_roll']==1].tolist()
print(f'Rolls at indices: {rolls[:10]}')
for ri in rolls[:5]:
    if ri == 0: continue
    c_before = float(df['close'].iloc[ri-1])
    c_after = float(df['close'].iloc[ri])
    pips = (c_after - c_before) / 0.0001
    mark = '□' if abs(pips) < 200 else '△□'
    print(f' {mark} roll @ idx={ri}: {c_before:.5f} -> {c_after:.5f} ({pips:+.1f} pips)')

```

(E) Label sanity — events have non-zero direction

(هـ) فحص ال events — labels لها direction غير صفر

```

python -c "
import pandas as pd
df = pd.read_parquet('combined_6y/day_trading_features.parquet')
n = len(df)
events = (df['event_flag'] == 1).sum()
event_dir_dist = df.loc[df['event_flag']==1, 'event_direction'].value_counts().to_dict()
print(f'Total bars: {n:,}')
print(f'Events: {events:,} ({100*events/n:.2f}%)')
print(f'Event direction distribution: {event_dir_dist}')
# Sanity: most events should have non-zero direction
zero_dir = event_dir_dist.get(0, 0)
assert zero_dir / max(events, 1) < 0.05, f'{zero_dir} events have direction=0 (>5%)'
print('□ Most events have non-zero direction')

```

(F) Train / holdout slice has both classes

(و) train + holdout كلاهما يحوي events

```

python -c "
import pandas as pd
df = pd.read_parquet('combined_6y/day_trading_features.parquet')
if 'dataset_slice' in df.columns:
    for sl in ('train', 'holdout'):
        sub = df[df['dataset_slice'] == sl]
        ev = (sub['event_flag'] == 1).sum()
        print(f'{sl}: {len(sub):,} bars | {ev:,} events')
else:
    print('dataset_slice column missing — pipeline will use default 75/25 split')

```

(G) NO duplicate timestamps

(ز) لا توجد ts مكررة

```
python -c "  
import pandas as pd  
df = pd.read_parquet('combined_6y/day_trading_features.parquet')  
dup = df['ts_event'].duplicated().sum()  
print(f'Duplicate timestamps: {dup}')  
assert dup == 0, f'{dup} duplicate timestamps – combine_quarter_artifacts has a bug'  
print('✅ No duplicates')  
"
```

7. Step 6 — SSL Pretraining

٧. الخطوة ٦ — التدريب السببي SSL

Goal

الهدف

Pretrain HierarchicalLOBTransformer + PriceCycleModel on the combined 6-year dataset, then extract 161-dim embeddings per bar.

تدريب HierarchicalLOBTransformer على dataset الـ ٦ سنوات، ثم استخراج 161-dim embeddings لكل bar.

```
./self_supervised/run_ssl_only.sh combined_6y checkpoints/ssl_6y 0.75
```

```
# Outputs (~5-8 hours on GPU):
```

```
# checkpoints/ssl_6y/lob/best_ssl_lob.pt      ← LOB transformer
# checkpoints/ssl_6y/cycle/best_ssl_cycle.pt  ← Price cycle
# checkpoints/ssl_6y/embeddings/embeddings.npy ← (145000, 161)
# checkpoints/ssl_6y/embeddings/embedding_valid.npy ← bool mask
```

Quality checks:

فحوصات الجودة:

- Phase A (LOB) val loss decreased monotonically and reached ~0.5-0.8 final
- Phase B (Cycle) val loss reached ~2.0-2.5 final
- embedding_valid.npy.sum() / len(df) > 70% (with 6yr data, expect 80-90%)
- embeddings.npy shape is (N_bars, 161) and dtype float32
- No NaN in embeddings (np.isnan(emb).any() == False for valid rows)
 - نزول بانتظام ووصل ~٠.٥-٠.٨ Phase A (LOB) val loss
 - وصل ~٢.٠-٢.٥ Phase B (Cycle) val loss
 - (مع ٦ سنوات بيانات: ٨٠-٩٠% embedding_valid > ٧٠%)
- embeddings.npy شكله (N_bars, 161) و dtype float32
- مفيش NaN في embeddings للصفوف الصحيحة

8. Step 7 — Train Hybrid Model

٨. الخطوة ٧ — تدريب النموذج الهجين

Goal

الهدف

Train the HybridModel that fuses day_trade features + SSL embeddings into event/direction/confidence predictions.

الذي يدمج features الـ day_trade + embeddings الـ SSL إلى تنبؤات event/direction/confidence.
تدريب HybridModel

```
python -m modules.trading_intel.training.train_hybrid \
--features combined_6y/day_trading_features.parquet \
--ssl-embeddings checkpoints/ssl_6y/embeddings/embeddings.npy \
--embedding-valid checkpoints/ssl_6y/embeddings/embedding_valid.npy \
--output checkpoints/hybrid_6y \
--use-dataset-slice \
--epochs 80 \
--hidden-dim 256 \
--n-layers 3 \
--dropout 0.3 \
--lr 3e-4
```

Quality checks:

فحوصات الجودة:

- Train loss decreases monotonically; val loss plateaus then early-stops
- Val event_acc > 0.65 (random ≈ 0.05)
- Val dir_acc > 0.60 (random ≈ 0.33)
- predictions.parquet contains all expected columns (event_prob, p_long, p_short, p_neutral, confidence)
- best_hybrid.pt contains norm_daytrade, norm_ssl, norm_cnn (if CNN used)
- Train loss early stop يثبت ثم val loss ينزل بانتظام:
 - Val event_acc > ٠.٦٥ (عشوائي ≈ ٠.٠٥)
 - Val dir_acc > ٠.٦٠ (عشوائي ≈ ٠.٣٣)
- predictions.parquet يحتوي كل الأعمدة المتوقعة
- best_hybrid.pt يحوي norm_daytrade, norm_ssl, norm_cnn

9. Step 8 — Strict Backtest & Decision Policy

٩. الخطوة ٨ — Backtest صارم + سياسة القرار

Baseline strict backtest (rule pipeline only):

ال Baseline الصارم (قواعد فقط):

```
python self_supervised/backtest_day_trade.py \
  --features combined_6y/day_trading_features.parquet \
  --output backtests/baseline_6y_strict \
  --strict \
  --starting-equity 100000 \
  --contracts-per-trade 1
```

Expected: ~78% hit, ~3-6% annualized over the 6-year holdout.

Hybrid-enhanced backtest (rule + SSL + hybrid):

Backtest (قواعد الهجين + SSL + hybrid):

(After hybrid predictions are integrated as a filter / re-rank — see modules.trading_intel.hybrid.policy.decide() for the policy logic.)

Quality checks:

فحوصات الجودة:

- Baseline holdout hit rate within $\pm 3\%$ of expected (~75-81%)
- Hybrid hit rate $\geq$ baseline (otherwise the hybrid is hurting, not helping)
- Max drawdown $< 1.5\%$ (with 1 contract on \$100k)
- Sharpe (annualized, non-overlapping trades) is statistically significant given the trade count
- Trade count is plausible (~150-300 trades / year for 6B)
 - Baseline hit rate $\pm 3\%$ من المتوقع (~٧٥-٨١ holdout في ال)
 - Hybrid hit rate $\geq$ baseline (ولا ال hybrid يضر)
 - Max drawdown $< 1.5\%$ (مع) contract ١٠٠\$ ك
 - Sharpe (annualized) ذو معنى إحصائي مع عدد الصفقات
 - عدد الصفقات معقول (~١٥٠-٣٠٠ سنويا ل 6B)

10. Appendix — Troubleshooting

١٠. ملحق — استكشاف الأخطاء

Q: Extractor: 'no_rollover_picked_dominant' on most months

في معظم الشهور extractor: 'no_rollover_picked_dominant' ال: س

A: Likely cause: the file contains only ONE contract per month. This is normal for non-rollover months — the dominant contract is used as-is.

يستخدم كما هو — rollover السبب الأرجح: الملف يحوي عقد واحد فقط في الشهر. ده طبيعي لشهور غير ال: ج
العقد المسيطر.

Q: Extractor: applied_offset is NaN

extractor: applied_offset = NaN ال: س

A: Pre-rollover window had no overlap with both contracts. Check raw data — both contracts must have at least 3 days of overlap before the rollover_day.

من البيانات الخام — كلا العقدين يجب أن يكون لديهما ٣ أيام تداخل على الأقل قبل rollover نافذة ما قبل ال: ج
rollover. لم تتداخل مع كلا العقدين. تحقق

Q: combine_quarter_artifacts: shape mismatch

combine_quarter_artifacts: shape mismatch ال: س

A: Means LOB tensors / order tensors don't have the same first dim as the features parquet in one of the quarter dirs. Re-run prepare_day_trading + build_order_batches for that quarter with matching --freq / --lookback-bars / --n-orders settings.

المجلدات. أعد تشغيل features parquet عندها أول بعد مختلف عن ال: ج
LOB tensors / order tensors يعني
prepare_day_trading + build_order_batches للربع بنفس ال: ج
--freq / --lookback-bars /
--n-orders.

Q: SSL embedding_valid is too low (< 50%)

SSL embedding_valid (> ٥٠) منخفض ال: س

A: Likely cause: too many session_break markers, possibly because the bar duration < daily maintenance gap. With 15min bars, expect ≥80% valid. If lower, check is_session_break distribution.

توزيع bar duration < maintenance gap كثيرة جدا، ربما بسبب session_break السبب الأرجح: علامات ج
is_session_break اليومي. مع شموع ١٥ دقيقة، توقع >= ٨٠٪ صالحة. لو أقل، تحقق من

Q: Hybrid model val accuracy plateaus near random

يثبت قرب العشوائي Hybrid val accuracy ال: س

A: Likely cause: SSL embeddings have collapsed during pretraining OR feature columns include leakage. Check that LEAKAGE_PATTERNS in train_hybrid.py covers all label-derived columns.

تحقق أن leakage تحوي features أو، pretraining انهارت في SSL embeddings: السبب الأرجح ج
labels. يغطي كل الأعمدة المشتقة من ال: ج
train_hybrid.py في LEAKAGE_PATTERNS

Q: Backtest hit rate << 78%

%أقل بكثير من ٧٨ Backtest hit rate :س

A: Likely cause: not running in --strict mode, OR holdout split is different from training.
Always backtest with --strict and verify split boundaries.

من حدود الـ --strict مختلف عن التدريب. دائما شغل holdout أو الـ --strict mode، السبب الأرجح: لم تشغل :ج
split. وتحقق

For deeper architecture details, see SUBSYSTEMS.md and modules/trading_intel/README.md.

للمزيد من تفاصيل المعمارية، راجع SUBSYSTEMS.md و modules/trading_intel/README.md.